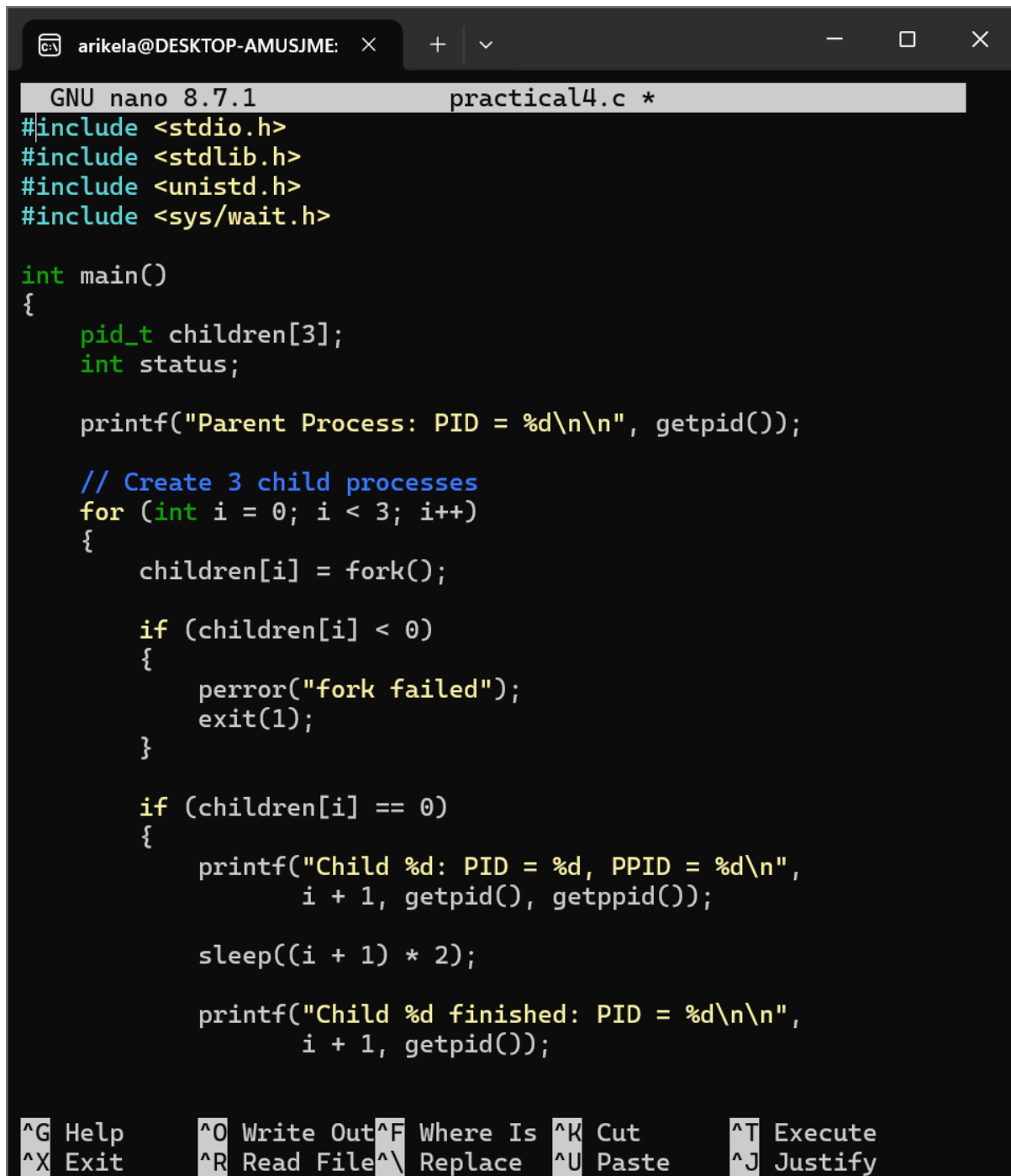


Practical Week -4

Task-1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.



The screenshot shows a terminal window with a dark background. At the top, there's a tab labeled 'arikela@DESKTOP-AMUSJME:'. Below the tab, the terminal title bar says 'GNU nano 8.7.1 practical4.c *'. The main area contains C code for a program named 'practical4.c'. The code includes headers for stdio, stdlib, unistd, and sys/wait. The main function starts by printing the parent's PID. It then creates three child processes using fork(). Each child process prints its own PID and PPID, sleeps for a duration proportional to its index (0, 2, and 4 seconds respectively), and then prints a message indicating it has finished. The code uses printf for all output. At the bottom of the terminal, there is a row of keyboard shortcuts: ^G Help, ^O Write Out, ^F Where Is, ^K Cut, ^T Execute, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, and ^J Justify.

```
GNU nano 8.7.1 practical4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t children[3];
    int status;

    printf("Parent Process: PID = %d\n\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0)
        {
            printf("Child %d: PID = %d, PPID = %d\n",
                   i + 1, getpid(), getppid());

            sleep((i + 1) * 2);

            printf("Child %d finished: PID = %d\n\n",
                   i + 1, getpid());
        }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
        sleep((i + 1) * 2);

        printf("Child %d finished: PID = %d\n\n",
               i + 1, getpid());

        exit(0);
    }
}

// ----- WAIT() -----
printf("Parent using wait()...\n");

for (int i = 0; i < 2; i++)
{
    pid_t finished = wait(NULL);

    printf("wait(): Child PID %d completed.\n", finished);
}

// ----- WAITPID() -----
printf("\nParent using waitpid()...\n");

// Wait for the specific remaining child
waitpid(children[2], &status, 0);

printf("waitpid(): Specific Child PID %d completed.\n",
       children[2]);

printf("\nAll child processes have completed.\n");

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
arikel@DESKTOP-AMUSJME: ~$ nano practical4.c
arikel@DESKTOP-AMUSJME: ~$ gcc practical4.c -o practical4
arikel@DESKTOP-AMUSJME: ~$ ./practical4
Parent Process: PID = 501

Child 1: PID = 502, PPID = 501
Child 2: PID = 503, PPID = 501
Parent using wait()...
Child 3: PID = 504, PPID = 501
Child 1 finished: PID = 502

wait(): Child PID 502 completed.
Child 2 finished: PID = 503

wait(): Child PID 503 completed.

Parent using waitpid()...
Child 3 finished: PID = 504

waitpid(): Specific Child PID 504 completed.

All child processes have completed.
arikel@DESKTOP-AMUSJME: ~$ |
```

Task-2

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

arikela@DESKTOP-AMUSJME: ×

+ v

— □ ×

GNU nano 8.7.1

zombie.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());

        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Parent does not call wait()
        printf("Parent sleeping for 30 seconds...\n");
    }
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^ Replace

^K Cut

^U Paste

^T Execute

^J Justify

arikela@DESKTOP-AMUSJME: × + ▾ − □ ×

GNU nano 8.7.1zombie.c *

```
pid = fork();

if (pid < 0)
{
    perror("fork failed");
    exit(1);
}

if (pid == 0)
{
    // Child process
    printf("Child process started.\n");
    printf("Child PID: %d\n", getpid());

    printf("Child process terminating...\n");
    exit(0);
}
else
{
    // Parent process
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    // Parent does not call wait()
    printf("Parent sleeping for 30 seconds...\n");
    sleep(30);

    printf("Parent terminating.\n");
}

return 0;
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
arikel@DESKTOP-AMUSJME:~$ nano zombie.c
arikel@DESKTOP-AMUSJME:~$ gcc zombie.c -o zombie
arikel@DESKTOP-AMUSJME:~$ ./zombie
Parent PID: 554
Child PID: 555
Parent sleeping for 30 seconds...
Child process started.
Child PID: 555
Child process terminating...
Parent terminating.
arikel@DESKTOP-AMUSJME:~$ ps -o pid,ppid,state,cmd
  PID   PPID S  CMD
   310    309 S  -bash
   567    310 R  ps -o pid,ppid,state,cmd
arikel@DESKTOP-AMUSJME:~$
```

arikela@DESKTOP-AMUSJME: ×

+ v

— □ ×

GNU nano 8.7.1

zombie.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());

        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Parent does not call wait()
        printf("Parent sleeping for 30 seconds...\n");
    }
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^ Replace

^K Cut

^U Paste

^T Execute

^J Justify